

SparseGraphSage: A Graph Neural Network Approach for Corporate Credit Rating

Si Shi*

Macao Polytechnic University
Macao, China
p2009153@mpu.edu.mo

Rita Tse

Macao Polytechnic University
Macao, China
ritatse@mpu.edu.mo

Wuman Luo

Macao Polytechnic University
Macao, China
luowuman@mpu.edu.mo

Giovanni Pau

University of Bologna
Bologna, Italy
University of California
Los Angeles, USA
giovanni.pau@unibo.it

ABSTRACT

Corporate Credit Rating (CCR) remains a critical research problem. In the past few decades, various machine learning approaches have gradually replaced and surpassed traditional labor-consuming manual checking. In particular, Graph Neural Networks (GNNs) have shown their capabilities and potential due to their strong power of processing non-Euclidean data. However, the current GNNs methods have two issues: 1) the proper design and construction of graphs; 2) the slow running speed and vast consumption of computing power. To address these issues, we propose a method named ‘SparseGraphSage’, which incorporates randomness in graph construction and integrates diffusion and sparse techniques in the GraphSage model. We design a stochastic edge selection process in the construction stage and diffusion matrices acting as operators in the graph layers. Through sufficient experiments and ablation study on two open-source CCR datasets, we demonstrate that our method exceeds the current state-of-the-art GNNs baselines in performance and is proven efficient.

CCS CONCEPTS

• **Applied computing** → **Enterprise computing**; **Business process management**; **Business intelligence**.

KEYWORDS

corporate credit rating, graph diffusion, graph neural networks, machine learning

ACM Reference Format:

Si Shi, Wuman Luo, Rita Tse, and Giovanni Pau. 2024. SparseGraphSage: A Graph Neural Network Approach for Corporate Credit Rating. In *2024*

**Corresponding author

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ICSCA 2024, February 1–3, 2024, Bali Island, Indonesia

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0832-9/24/02

<https://doi.org/10.1145/3651781.3651800>

13th International Conference on Software and Computer Applications (ICSCA 2024), February 1–3, 2024, Bali Island, Indonesia. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3651781.3651800>

1 INTRODUCTION

Corporate Credit Rating (CCR) is a well-known financial risk management term that indicates the risk ranking of a specific company. Generally, it symbolizes the possibility that a corporation pays its financial obligations on time and legally conducts transactions with others [5]. CCR plays a significant role in both academia and industry. Conventionally, it takes a long duration for a ranking institute to obtain detailed ranking information manually. As a consequence, a number of rating institutes are seeking safer, more accurate and efficient approaches to predict credit rating.

With the proliferation of machine learning and artificial intelligence, a series of automatic approaches have emerged. Logistic Regression (LR), Support Vector Machine (SVM) and Neural Networks methods are proven to surpass manual auditing in speed and accuracy [8, 10, 12, 13]. Moreover, Graph Neural Networks (GNNs) methods show their superiority in processing non-structural and complex data [7, 9, 14, 15]. The graph-formed neural network layers are usually deep and have topological characteristics. The non-Euclidean structures enable them to capture the inner relationships among nodes and features better. Thus, the GNNs-related credit rating approaches have been applied in recent years and beaten the traditional machine learning methods in capability [4, 5, 11]. However, two major issues exist so far in GNNs-related CCR methods.

First, the graph construction of CCR data remains a challenge. Because of the traditional tabular forms of datasets, the proper definition of nodes, edges and their weights needs more thorough consideration.

Second, the current GNNs-based methods are time-consuming. Since graph computing is more complex and demands higher computing power than traditional neural networks, it costs more time to derive a specific prediction.

To address the above two issues, we propose a novel GNNs approach: **SparseGraphSage**. In our method, we create a novel graph construction process to resolve the first issue. We incorporate randomness in edge selection particularly. In addition, we design a diffusion term and combine it with GraphSage [7] framework. This

procedure reduces the computing time to resolve the second issue. We implement our methods in two open-source CCR datasets. The abundant experimental results show that our method exceeds the state-of-the-art GNNs baselines regarding model performance. In addition, it costs little time to train our method. Among the GNNs methods, it is the fastest regarding the second dataset, and the second fastest regarding the first dataset. We also provide a brief time complexity analysis.

We summarize our main contributions below.

1. We design a novel graph construction pipeline under CCR domain and adds randomness in edge selection.
2. We incorporate diffusion techniques and create a GNNs approach, SparseGraphSage, that is quite efficient.
3. We demonstrate our method—SparseGraphSage—is superior to other state-of-the-art GNNs-related baselines.

2 RELATED WORK

2.1 Traditional Machine Learning Methods for CCR

In recent decades, many scholars have employed conventional machine learning methods for CCR. Some early approaches have a close connection with statistics. For instance, Erkki K. Laitinen [10] estimated corporate credit analysts' risk by deploying LR and linear regression models. As time passes, various models like SVM and their variants have shown their effectiveness. Maldonado et al. [13] utilized two SVM variants in order to derive the best feature selection of credit loan data by considering the inner relationship between variable interactions. Also, Kyoung-jae Kim et al. [8] proposed a multi-class SVM by deploying an ordinal pairwise partitioning (OPP) strategy. Neural Networks-based approaches outperform their predecessors. Cuicui Luo et al. [12] proposed a deep belief network to predict credit rating. It outperforms SVM and LR.

These methods are applicable whereas they have limitations like insufficient ability to process data with nonlinear distribution. Therefore, more novel neural network methods emerged.

2.2 GNNs-Related Methods for CCR

GNNs-related methods originate from traditional neural networks but have an enhanced ability to process non-Euclidean data. The versatile styles of GNNs designs increase the species and implementation of CCR problem. Bojing Feng et al. [5] proposed CCR-GNN. They created a corporate-to-graph structure and established multiple graphs with different companies. This design is innovative and retains the original feature information to the largest extent. Nevertheless, the complexity of algorithms makes the training process longer. Junyi Liu et al. [11] proposed SA-GNN for inter-bank credit rating attacks defending. This research is valuable. However, the model has limitations such as low migration and restrictions on datasets because of the sparsity and privacy of attack data.

To sum up briefly, GNNs-related methods generally outperform traditional machine learning methods, but they still need further improvement in efficiency and feasibility. These facts inspire us to conduct this research.

3 METHODOLOGY

We formulate our research problem and depict the whole model structure and algorithm in the following subsections.

3.1 Problem Formulation

Let $M = \{M_1, M_2, \dots, M_n\}$ denotes the original corporate dataset, where $M \in \mathbb{R}^d$, d represents the number of features for each corporate, n denotes the number of corporates.

We assume an undirected homogeneous graph $G = \{N_i, E_j, A\}$, where $N_i (i \in n)$ denote nodes, E_j denote edges, and $A \in \mathbb{R}^{n \times n}$ denotes the adjacency matrix of G .

Let $M \rightarrow G$, our target is to obtain the label set $T = \{t_1, t_2, \dots, t_n\}$, which denotes the final credit rating results for each corporation. In this paper, $t_i = \{0, 1, 2, \dots, p\} (p \geq 2)$.

3.2 Model Framework

Our model includes three stages, i.e., graph construction, main graph layers building and final estimation. The general overview is provided in Fig. 1.

3.2.1 Graph Construction. First, we conduct data preprocessing. We apply a combination of universal preprocessing method and expert participation based on the different dataset structures. We remove useless features such as company id. Then, we employ standardization and normalization to ensure the consistency of quantity level for numerical features. As to categorical features, we transform them into numerical ones with specific functions. The details are depicted in Section 4.2. After preprocessing the data, we transform the original tabular dataset M into an entire homogeneous graph $G = \{N_i, E_j\}$.

Each node N_i denotes a single company, the node features denote the feature variables of M , labels are turned into node label L . Each edge E_j represents the cosine similarity between two connected nodes, this similarity value is stored as the weight for every edge.

For each node, we randomly select $m (m < n)$ connections and delete the duplicated l edges. The duplicated edges denote the edges with the same vertices because the random number can be generated repeatedly. Therefore, the total edge number will diminish while still retaining the most significant information. Through this stochastic selection procedure, the complexity of the problem will decrease. The number of edges is $mn - l$. We define the procedure in (1) and (2).

$$N_i = \{M_1, M_2, \dots, M_n\}, i = \{1, 2, \dots, n\}, \quad (1)$$

where n denotes the number of companies.

$$E_j = \frac{N_p \cdot N_q}{\|N_p\| \times \|N_q\|}, j = \{1, 2, \dots, mn - l\}, \quad (2)$$

where m denotes a random selected number, l represents the number of duplicated edges.

3.2.2 Diffusion Layers and GraphSage Layer. We construct main graph layers: Diffusion and GraphSage layers below. We provide detailed explanation in Algorithm 1.

Through the graph construction, we derive Graph G and related matrices.

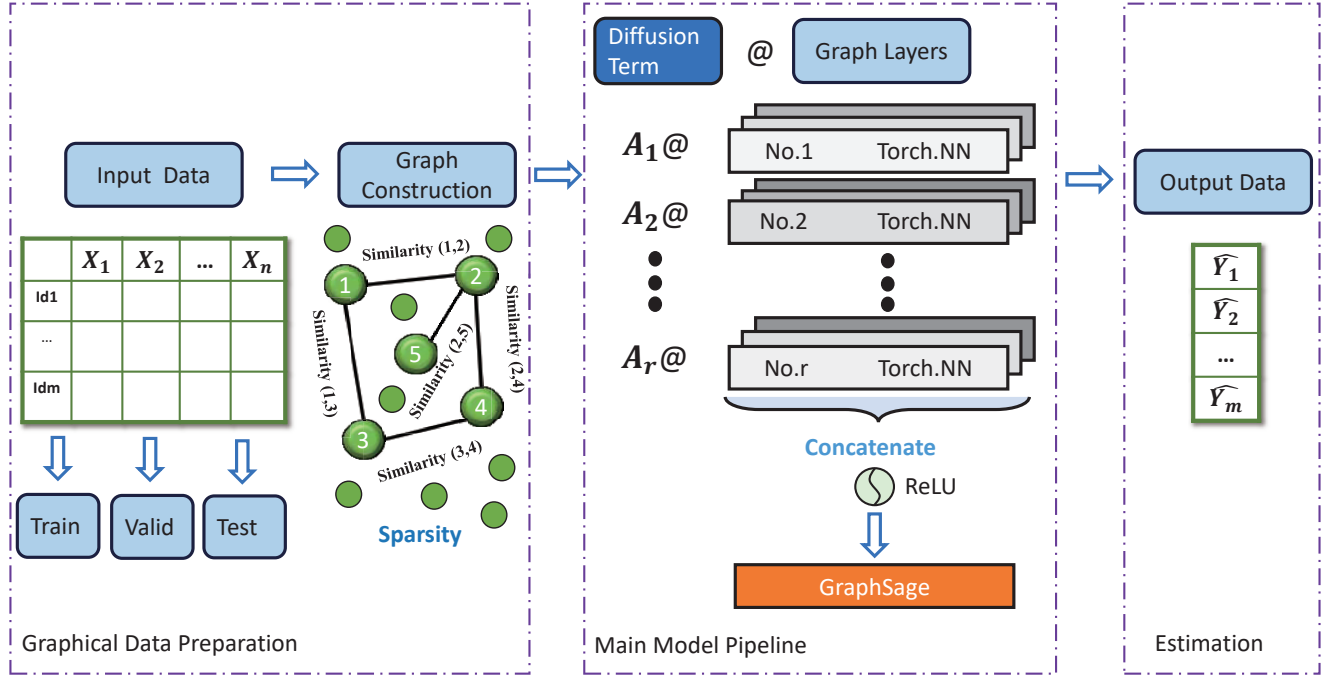


Figure 1: Method Framework Overview. We first preprocess and split the original inputs. Then, we construct graph data by incorporating randomness. We import the data into the concatenation of diffusion matrices and linear graph layers. The changing grey color symbolizes the dissemination of information among graph layers. After adding a GraphSage layer, we derive the final estimation targets.

Algorithm 1 Diffusion Layers and GraphSage Layer.

Input: Graph $G(N, E)$; r : hop number; k : layer depth; σ : nonlinear activation function.

- 1: SPARSE DIFFUSION GRAPH:
- 2: Derive A : adjacency matrix, I : identity matrix, D : diagonal node degree matrix of $(A + I)$ from G .
- 3: **for** $p = 0$ to r **do**
- 4: $A_p \leftarrow D^{-\frac{1}{2}}(A + I)D^{-\frac{1}{2}}$.
- 5: $S \leftarrow \sigma([N\theta_0, A_1N\theta_1, \dots, A_pN\theta_p])$.
- 6: **end for**
- 7: $Z \leftarrow \text{ReLU}(S, \text{dim} = 1)$.
- 8: GRAPH SAGE:
- 9: **for** $q = 1, 2, \dots, k$ **do**
- 10: $H_q \leftarrow \text{MEANAGGREGATE}\{Z\}$.
- 11: **end for**
- 12: $H_{all} \leftarrow \sigma(\text{CONCAT}(H_k))$.

Output: H_{all}

In light of Graph Diffusion [6] techniques, We utilize multiple diffusion matrices A_r to incorporate sparsity into linear graph layers. The diffusion matrix transmits information from one node to another by acting as an operator. Through this dissemination

process, the overall training will speed up because of the sparse characteristics of this matrix.

Then, we deploy activation functions to concatenate all the above layers and derive layer S . $N \in \mathbb{R}^{n \times d}$ is a node feature matrix, θ_r are learnable weights, '...' denotes concatenation.

We add a ReLU function to S , preparing for the next graph formation layers. The layer Z remarks the summarising of diffusion layers and output dimension deduction.

During the second stage, we apply a GraphSage Layer H_{all} to the above output Z .

The core principle of GraphSage is to sample each neighbor vertex and aggregate feature information from neighbors and derive estimation [7]. In this paper, we apply *MEANAGGREGATE* method as the option for aggregation. It indicates the mean approach for each tensor dimension. Through this aggregation method, we obtain H_k as aggregated neighborhood vector.

Finally, we concatenate H_k and apply a fully connected layer with activation function σ . Thus, we derive H_{all} as the constructed GraphSage Layer.

3.2.3 Estimation. We obtain the output target $\hat{Y}$ by dividing the L^2 norm of H_{all} . The details is provided in (3).

$$\hat{Y} = H_{all} / \|H_{all}\|_2. \quad (3)$$

Table 1: Description of Dataset 1

Item	Content
Number of instances	5000
Number of classes	10 (from AAA to BBB-) (COMMEQTA, LLPLOANS, COSTTOINCOME, ROE, LIQASSTA, SIZE)
Number of features	6
Label	'rating'

Table 2: Description of Dataset 2

Item	Content
Number of instances	2029
Number of classes	3 (0 represents AAA, AA, A; 1 represents BBB; 2 represents others)
Number of features	30 (Debt Ratios, Cash Flow Indicator Ratios, etc.)
Label	'Rating'

4 EXPERIMENTS

4.1 Experimental Setup

We implement all of the experiments on a Linux server and a Windows 10 laptop. The server contains an NVIDIA RTX A6000 Graphic Card with 48G as its memory. The devices of the Windows are an Intel(R) Core(TM) i5-1035G4 CPU and 8 GB RAM. We programmed with Python 3.8, Scikit-learn machine learning library, TensorFlow 2.0 framework, dgl 1.1.0 library and PyTorch 2.0.0 framework with CUDA 11.7.

4.2 Datasets

We deploy two open-source datasets. Dataset 1 originates from Credit Risk Analytics book by Bart Baesens, Daniel Roesch, Harald Scheule¹. It contains 5000 instances, each representing a company. The original dataset has 7 features and 1 label with 10 classes. After removing a useless feature 'spid' which denotes company id, we obtain the final dataset. A brief description is given in Table 1.

The 10 classes have equal amounts, and the 6 features are specific financial indicators.

Dataset 2 originates from a Kaggle dataset repository². It includes 2,029 rating results ranked by the major rating institutes. There are 30 features covering 25 financial statements and 5 demographic and time details. The financial indicators are numerical, while the others are in categorical form. We apply LabelEncoder() transformation and change them to numerical data. The original dataset contains 10 classes from AAA to D. We divide the labels into 3 classes for the sake of preventing imbalance. The details are depicted in Table 2.

¹<https://github.com/Polanitz/Multiclass-Classification-for-Corporate-Credit-Ratings/blob/main/ratings.csv>

²<https://www.kaggle.com/agewerc/corporate-credit-rating>

After preprocessing data, '0' labels have 494 items, '1' labels have 671 items, and '2' labels have 864 items. They have comparatively even distribution; thus, no imbalance problem exists.

4.3 Evaluation Metrics

We provide detailed formulas below and depict four metrics we apply in our research.

Accuracy (ACC) is denoted by the correctly classified instances divided by the total instances. It can be expressed in (4):

$$ACC = \frac{RP + RN}{RP + FP + FN + RN}, \quad (4)$$

where RP represents real positive, RN stands for real negative, FP refers to false positive, FN denotes false negative.

Recall [2] stands for the proportion of real positive instances correctly identified by the method. Its calculation is provided in (5):

$$RECALL = \frac{RP}{RP + FN}, \quad (5)$$

Precision [2] evaluates the proportion of correctly predicted positive cases. Its calculation is provided in (6):

$$PRECISION = \frac{RP}{RP + FP}, \quad (6)$$

F1-Score [2] measures the harmonic mean of precision and recall. It has different variants, we apply Macro-F1-Score, which applies the mean of F1-scores of each classification label in our paper. The common form of F1-Score calculation method is stated in (7):

$$F1 = \frac{2 * PRECISION * RECALL}{PRECISION + RECALL}. \quad (7)$$

4.4 Baselines

We list some representative methods as our baselines.

- LR[3] is a conventional machine learning classification algorithm. It estimates the probabilities of occurrence of an event based on given independent variables.
- Graph Convolutional Networks (GCN)[9] symbolizes the revolution of deep learning landscape by transmitting convolution computing techniques to graph-formed data. It represents the beginning of GNNs.
- Graph Attention Networks (GAT)[15] aggregates neighbor nodes by self-attention mechanism and achieves self-adaptive matching.
- GraphSage [7] provides the graph data with inductive learning capabilities by random sampling and different aggregations. Its most extraordinary advantage is the fast computing speed.
- CCR-GNN [5] is one of the representative state-of-the-art GNNs-based CCR methods. The creation of corporate-to-graph design is innovative.

4.5 Results

After dataset preprocessing and standardization of features, we split the input data into training, validation and testing subsets according to the ratio 6:2:2. Then, we apply our main method architecture and compare it with GNNs baselines. We apply CrossEntropy() as loss function, and Adam as our optimizer. The training epochs are set as

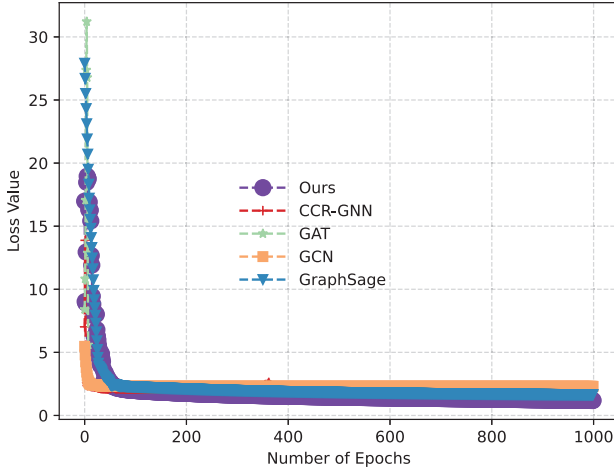


Figure 2: Historical Loss Values of Dataset 1.

3000 for Dataset 1 and 800 for Dataset 2. The results are displayed below.

4.5.1 Performance Comparison. We provide performance comparison with the baselines based on four metrics and training time. The average performance results with errors are expressed in Table 3.

The results show that our method surpasses other baselines in accuracy and precision. Note that LR is the fastest, but the performance is not satisfactory. CCR-GNN takes a long training time. GraphSage method has the second fastest training time, and our model is the third fastest regarding this dataset. The possible reason is the simple structure of Dataset 1 which only contains 6 numerical features. This fact proves the efficiency of GraphSage to process simple data. However, the average capabilities of GraphSage are weaker than our method, which demonstrates the effectiveness of the proposed approach.

As for Dataset 2, we elaborate the results in Table 4 similarly.

Table 4 embodies that our model achieves the best performance and takes the least training time among GNNs methods. LR is the fastest, but the performance is suboptimal. Compared with the performances of Dataset 1, it shows that our model has a stronger ability and efficiency in processing more complex data.

4.5.2 Loss Visualization. We plot the historical loss values of all models as illustrations. We select the first 1000 and 80 epochs as ranges to display the losses in their most clear forms. The results are visualized as Fig. 2 and Fig. 3.

Fig. 2 shows that during the training stages of Dataset 1, all of the losses have similar declining patterns, which are consistent with the computing theories. Note that our model has the least losses during all the training epochs. It proves the effectiveness of this method.

Fig. 3 shows a phenomenon that is slightly different from Dataset 1. The model GAT has a dramatically high loss during the first 20 epochs, and the losses decline rapidly after that, which demonstrates the traditional graph attention mechanism is trainable but is not

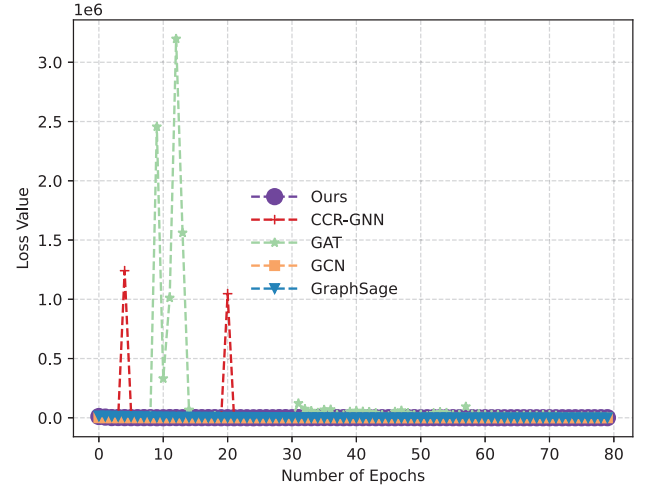


Figure 3: Historical Loss Values of Dataset 2.

quite suitable for this task. All of the other baselines show low losses.

4.5.3 Complexity Analysis. We provide a brief time complexity analysis of GNNs models and our proposed method.

Given a graph with L layers, N nodes, E edges, F node features, r neighbor nodes, and h dimensional node hidden features, GCN has time complexity $O(LN^2F + LNF^2)$, GAT has time complexity $O(LN^2F + LNF^2)$ which is equal to GCN [1], and GraphSage has time complexity $O(r^L N h^2)$ [16]. L layers of neural networks have complexity $O(LN^2F)$. Because of the integrated diffusion term of our method, the complexity of linear networks will reduce to $O(L|E|F)$. Therefore, the time complexity of our method is $O(L|E|F + rNh^2)$, which is the least. This fact is also proven by the experimental results.

4.5.4 Ablation Study. We present the ablation study results regarding two datasets in Table 5.

Table 5 shows the effectiveness of both GraphSage and Diffusion layers of our method. Neither GraphSage nor Diffusion alone can surpass the model with both components.

5 CONCLUSION

In conclusion, we propose a novel GNN method to resolve graph construction and efficiency problems. In particular, we incorporate randomness and diffusion techniques in this model. It is demonstrated to be effective and efficient experimentally and theoretically.

Due to data privacy, corporate credit rating data are generally not easily accessed. In our future work, data augmentation may be a direction. Also, integrating large language models with textual information included in GNNs methods can be promising.

ACKNOWLEDGMENTS

Our work is supported by the Macao Polytechnic University–Edge Sensing and Computing: Enabling Human-centric (Sustainable)

Table 3: Model Performance of Dataset 1

Metrics	Method					
	LR[3]	GraphSage[7]	GCN[9]	GAT[15]	CCR-GNN[5]	<i>SparseGraphSage (Ours)</i>
ACC	0.3380 ± 0.01	0.5640 ± 0.02	0.096 ± 0.001	0.096 ± 0.001	0.2230 ± 0.01	0.6700 ± 0.02
Recall	0.3375 ± 0.01	0.5645 ± 0.02	0.1 ± 0.001	0.1 ± 0.001	0.2404 ± 0.02	0.6612 ± 0.02
Precision	0.3282 ± 0.01	0.5628 ± 0.02	0.096 ± 0.001	0.097 ± 0.001	0.3014 ± 0.05	0.6625 ± 0.02
F1-Score	0.2930 ± 0.02	0.5587 ± 0.01	0.098 ± 0.005	0.098 ± 0.005	0.2 ± 0.02	0.6573 ± 0.02
Training time (seconds)	0.1599 ± 0.01	805.92 ± 5.00	2961.50 ± 10.00	33792.14 ± 10.00	21201.81 ± 10.00	842.36 ± 5.00

Table 4: Model Performance of Dataset 2

Metrics	Method					
	LR[3]	GraphSage[7]	GCN[9]	GAT[15]	CCR-GNN[5]	<i>SparseGraphSage (Ours)</i>
ACC	0.3910 ± 0.0080	0.4543 ± 0.02	0.3219 ± 0.02	0.3220 ± 0.02	0.3220 ± 0.05	0.5800 ± 0.02
Recall	0.3474 ± 0.01	0.3404 ± 0.01	0.3333 ± 0.01	0.3333 ± 0.01	0.3333 ± 0.01	0.4740 ± 0.001
Precision	0.3539 ± 0.01	0.4225 ± 0.02	0.1073 ± 0.01	0.0290 ± 0.005	0.0300 ± 0.005	0.4230 ± 0.02
F1-Score	0.3036 ± 0.01	0.1010 ± 0.02	0.1121 ± 0.01	0.0540 ± 0.005	0.0542 ± 0.005	0.3820 ± 0.01
Training time (seconds)	0.2344 ± 0.01	1104.09 ± 10.00	2935.06 ± 10.00	7734.14 ± 10.00	2164.35 ± 5.00	877.17 ± 5.00

Table 5: Ablation Study Results

Dataset	Method		Metrics			
	GraphSage	Diffusion	ACC	Precision	F1-Score	Recall
Dataset 1	✓	✗	0.5640	0.5628	0.5587	0.5645
	✗	✓	0.6030	0.5841	0.5737	0.5795
	✓	✓	0.6700	0.6625	0.6573	0.6612
Dataset 2	✓	✗	0.4543	0.3248	0.1010	0.3404
	✗	✓	0.4472	0.4143	0.1121	0.3544
	✓	✓	0.5800	0.4230	0.3820	0.4740

Smart Cities (RP/ESCA-01/2020). We appreciate Macao government for supporting this work.

REFERENCES

- [1] Derrick Blakely, Jack Lanchantin, and Yanjun Qi. 2021. Time and space complexity of graph convolutional networks. *Accessed on: Dec 31 (2021)*.
- [2] Nancy Chinchor and Beth M Sundheim. 1993. MUC-5 evaluation metrics. In *Fifth Message Understanding Conference (MUC-5): Proceedings of a Conference Held in Baltimore, Maryland, August 25-27, 1993*.
- [3] David R Cox. 1958. The regression analysis of binary sequences. *Journal of the Royal Statistical Society Series B: Statistical Methodology* 20, 2 (1958), 215–232.
- [4] Sanjiv Das, Xin Huang, Soji Adeshina, Patrick Yang, and Leonardo Bachega. 2023. Credit Risk Modeling with Graph Machine Learning. *INFORMS Journal on Data Science* (2023).
- [5] Bojing Feng, Haonan Xu, Wenfang Xue, and Bindang Xue. 2022. Every Corporation Owns Its Structure: Corporate Credit Rating via Graph Neural Networks. In *Chinese Conference on Pattern Recognition and Computer Vision (PRCV)*. Springer, 688–699.
- [6] Johannes Gasteiger, Stefan Weißenberger, and Stephan Günnemann. 2019. Diffusion improves graph learning. *Advances in neural information processing systems* 32 (2019).
- [7] Will Hamilton, Zhitaoying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. *Advances in neural information processing systems* 30 (2017).
- [8] Kyoung-jae Kim and Hyun-chul Ahn. 2012. A corporate credit rating model using multi-class support vector machines with an ordinal pairwise partitioning approach. *Computers & Operations Research* 39, 8 (2012), 1800–1811.
- [9] Thomas N Kipf and Max Welling. 2016. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907* (2016).
- [10] Erkki K Laitinen. 1999. Predicting a corporate credit analyst’s risk estimate by logistic and linear models. *International review of financial analysis* 8, 2 (1999), 97–121.
- [11] Junyi Liu, Dawei Cheng, and Changjun Jiang. 2023. Preventing attacks in interbank credit rating with selective-aware graph neural network. In *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence*. 6085–6093.
- [12] Cuicui Luo, Desheng Wu, and Dexiang Wu. 2017. A deep learning approach for credit scoring using credit default swaps. *Engineering Applications of Artificial Intelligence* 65 (2017), 465–470.
- [13] Sebastián Maldonado, Juan Pérez, and Cristián Bravo. 2017. Cost-based feature selection for support vector machines: An application in credit scoring. *European Journal of Operational Research* 261, 2 (2017), 656–665.
- [14] Franco Scarselli, Marco Gori, Ah Chung Tsoi, Markus Hagenbuchner, and Gabriele Monfardini. 2009. The Graph Neural Network Model. *IEEE Transactions on Neural Networks* 20, 1 (2009), 61–80. <https://doi.org/10.1109/TNN.2008.2005605>
- [15] Petar Velickovic, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, Yoshua Bengio, et al. 2017. Graph attention networks. *stat* 1050, 20 (2017), 10–48550.
- [16] Yu Xie, Maoguo Gong, Yuan Gao, AK Qin, and Xiaolong Fan. 2020. A multi-task representation learning architecture for enhanced graph classification. *Frontiers in neuroscience* 13 (2020), 1395.